Bridge Zero

Portaldot, proven on Ethereum and EVM chains.

A Portaldot light client on Ethereum and EVM chains.
Live on Sepolia, Base Sepolia, Arbitrum Sepolia and the BNB Smart Chain (BSC) testnet.

Portaldot Hacker House 2026 · Track 2 · Multi-Chain & Cross-Chain Infra

jonathan-moore58.github.io/bridge-zero/app

The plan on one page

LIVE ON TESTNETS

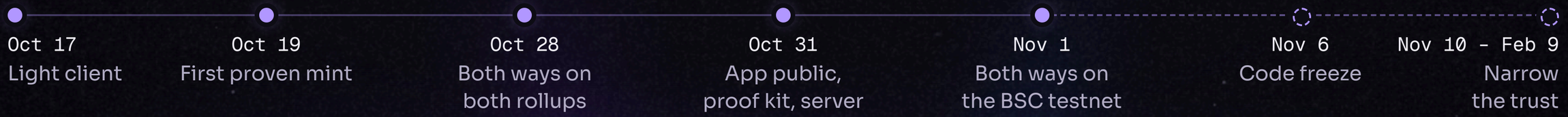
- The app
- The proof kit and data bridge
- Operations on a server

PROVEN ON CHAIN

- Portaldot's GRANDPA signatures
- Each lock's storage proof
- The commitment, each nonce once

TRUSTED TODAY

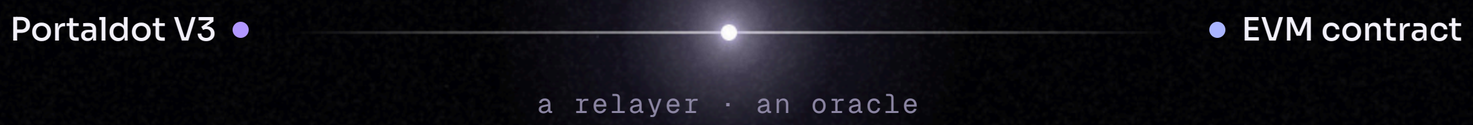
- V3's 2 validators and its sudo key
- Each light client's admin and checkpoint
- The way back: a 2-of-3 committee
- Vendored crypto and trie code, unaudited



EVM contracts cannot check Portaldot themselves

POT and Portaldot's state live on Portaldot. EVM wallets, apps and liquidity live on Ethereum, its rollups and other EVM chains.

Out of the box, a contract there cannot check Portaldot, so it takes someone's word for it: a relayer, an oracle.



Bridge Zero: the contract checks Portaldot's own finality and storage.

Our way back still takes a 2-of-3 committee's word. Slide 7 says how far.

How it works: lock, prove, mint



Anyone can relay or mint. A relayer that stops can delay a mint, not forge one.

2,391,069 gas
lock 1's mint · 14.3% of Sepolia's per-transaction cap

More than two thirds of the set must sign: 2 of 2 on V3, 7 of 10 on the old mainnet

ed25519 checked in Solidity, about 0.77M gas a signature

BLAKE2b through the BLAKE2F precompile

Substrate trie proofs down to a pallet-revive contract's slot

149 transactions on five chains

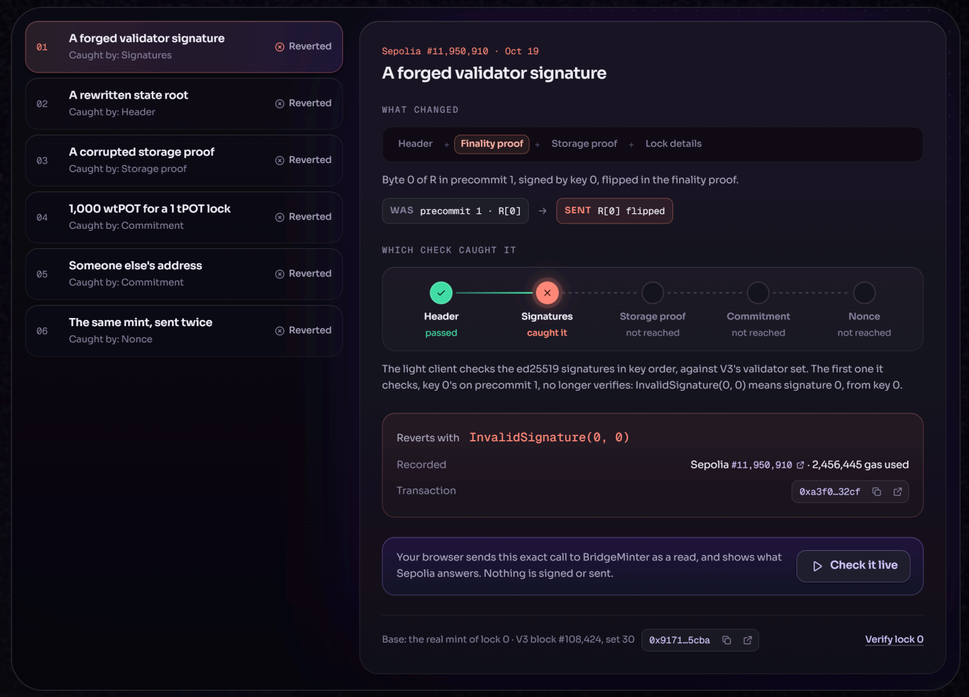
58 Sepolia · 25 Portaldot V3 · 24 Base Sepolia · 24 Arbitrum Sepolia · 18 BNB Smart Chain testnet

48 s — lock to mint, proven (lock 1, Oct 20): lock 10 · finality 14 · proofs 9 · mint 15

6 / 6 — attacks on a real mint reverted: forged signature · rewritten state root · corrupted proof · 1,000 wtPOT for a 1 tPOT lock · another recipient · replay

7 of 10 — the same code checked the old mainnet's real validator set (the V3 testnet's 2 validators are probably development keys)

THE APP'S ATTACK LAB · RECORDED ON SEPOLIA, OCT 19



- On Sepolia, every V3 validator-set change imported in order, set 19 to set 78 (Oct 17 – Nov 1)
- 370 forge · 403 relayer · 324 site tests · 432 fixture checks, in CI on every push
- Re-check the six attacks live: `jonathan-moore58.github.io/bridge-zero` · every row and its hash: `github.com/jonathan-moore58/bridge-zero`

The app: bridge it, then verify it

Lock tPOT on V3 for wtPOT on Sepolia, Base Sepolia, Arbitrum Sepolia or BSC testnet; burn it back (trusted).

WALLETS

MetaMask, or a Polkadot wallet for the lock: Portaldot Wallet, polkadot{.js}, SubWallet, Talisman.

AUTO-MINT (for locks of 0.001 tPOT or more)

You sign only the lock and need no ETH or tBNB on the way in. Locks **2**, **6**, **7** and **8** were minted **125** to **144** s after the lock, **90** s of it a grace after V3's finality; just after a V3 set change, a mint on a rollup or the BSC testnet can take up to about **20** min longer.

VERIFY

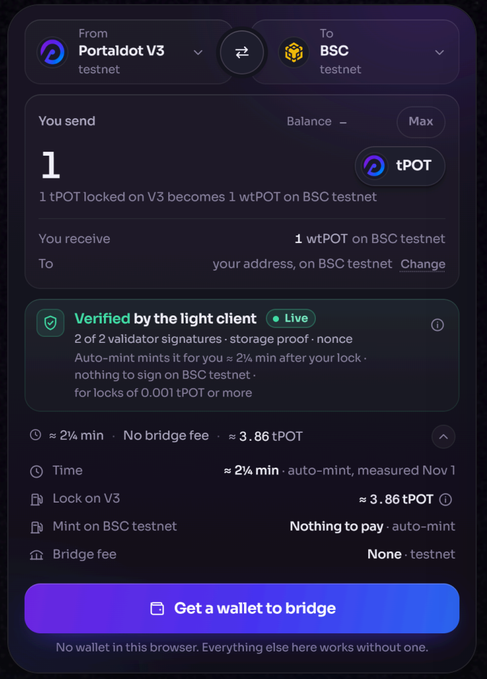
Pick a transfer: your browser checks V3's validator signatures, the lock's storage proof and the commitment, then the destination contract's own verdict.

Public since Oct 31. First transfers from the owner's own wallet on Oct 28, with the app run locally: both ways, both rollups.

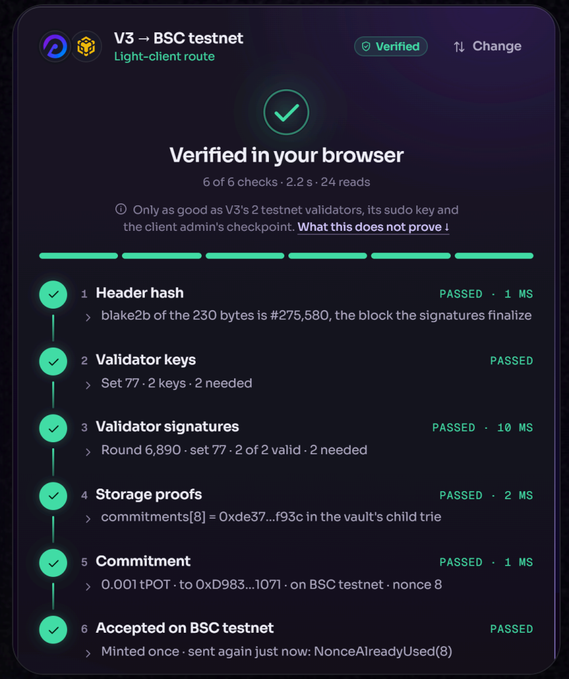
`jonathan-moore58.github.io/bridge-zero/app`

- Not run on chain yet: the fallback, a mint sent from your wallet with proofs built in the browser.

BRIDGE TAB · BSC TESTNET



VERIFY TAB · LOCK 8, NOV 1



- Lock 8 and its burn were sent from the command line.
- Not run on chain yet: a lock from a Polkadot wallet in the app (lock 3, the same call, was sent from the command line).

The way back, honestly

Burn wtPOT, and a 2-of-3 committee releases tPOT on V3. Nothing on V3 checks the burn.

On the testnet one operator holds all three keys, two of them on a cloud server since Oct 31.

Burn wtPOT ----> 2-of-3 committee ----> tPOT released on V3

STANDARD After the burn's chain is final, any amount.

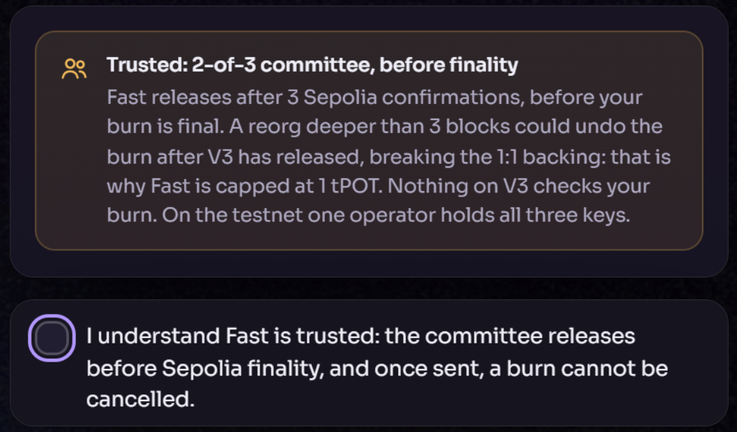
19 min **42** s for burn 0 from Sepolia, released by hand (Oct 20)
Sepolia's finality, as the watcher measured it: **12.5** to **19.9** min (median **16.0**)
46 s for burn 0 from the BSC testnet, which is final in about a second (Nov 1)

FAST At most 1 tPOT a burn; Sepolia and rollups share 1 tPOT in flight.

84 s from Sepolia (Oct 24) · **105** s from Arbitrum Sepolia (Oct 28)
905 s from Base Sepolia: held back by the shared budget, the limit working

- After row 149, Nov 1: vault outstanding **1.411** tPOT = wtPOT supply **1.411** on the four chains, no burn waiting
- Fast mode also trusts that a burn stays once 3 Sepolia blocks (36 s of a rollup's blocks) follow it.

THE APP, BEFORE A FAST BURN · NOV 1



The proof kit: Portaldot state for builders

PortaldotStateReader · Sepolia `0x2ea6…1fA9` · no owner, no admin, source-verified

READ	GAS ON SEPOLIA
An account's balance	452,621
An asset balance (pUSD)	480,180
An NFT's owner	363,589
A contract's storage slot	877,450

- The state root comes from the light client, never from the caller.
- Example: ProvenAssetGate (`0x24aB…8d23`) records holders of at least 1 pUSD at V3 **#264,729**.
- `npm run kit` fetches, walks and checks the proofs · **41** fixture checks pass

Solidity, through the Reader

```
// asset 7777 (pUSD) at a V3 block the light client imported
(uint128 balance, uint8 status, uint32 number) =
    reader.assetAccount(blockHash, 7777, holderId, proof);
// a block it did not import reverts UnknownBlock(hash)
```

Terminal: `npm run kit`, abridged from `docs/PROOF-KIT.md`

```
$ npm run kit -- read balance --account 0xD983…1071 \
    --from-fixture ../fixtures/v3-testnet-kit/state-reads.json
  B: V3 #264,729, imported by GrandpaLightClient #2 on Sepolia
  ok  off chain: free 159.47352201127852 tPOT
  ok  PortaldotStateReader.accountInfo returns the same
      AccountInfo at #264,729: 452,621 gas
Nothing was signed or sent.
```

- **v0.1 limits:** the Reader is on Sepolia only, the TypeScript is not an npm package yet, and no claim has been sent to the gate (no project account holds a V3 asset).
- **Trusted as for the bridge:** V3's validators, sudo key and runtime upgrades, the light client's admin and checkpoint, unaudited vendored code; for an asset such as pUSD, also its owner and admin, who can mint, burn and freeze it.

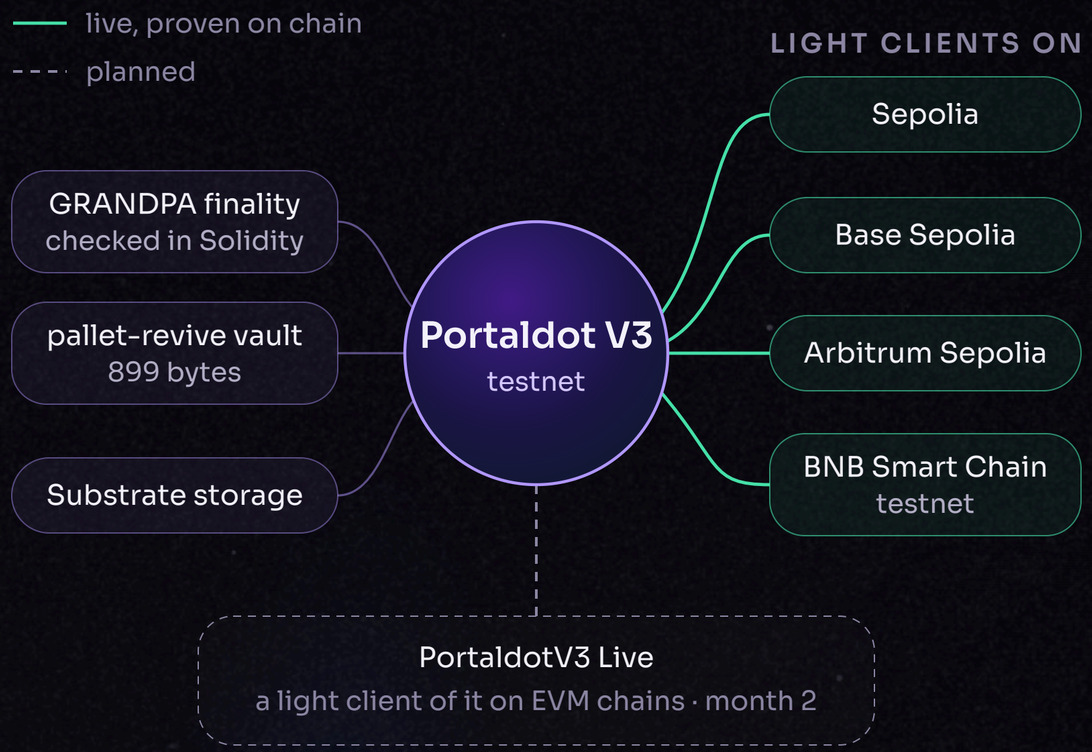
Why Portaldot is essential

WHAT PORTALDOT GETS

A path for EVM users and liquidity into POT: wtPOT on EVM chains, with a trusted way back, and proven Portaldot state in any EVM contract.

BUILT FROM PORTALDOT ITSELF

- On the way in, trust rests on Portaldot's validator set, with V3's sudo key and each light client's admin.
- Locks come from MetaMask or from a Portaldot account, the Portaldot Wallet path.
- One verifier, two Portaldot chains: the V3 testnet **2 of 2**, the old mainnet **7 of 10**.



Portaldot runs its own bridge between PortaldotV3 Live and BNB Smart Chain; iBridge is the cross-chain bridge in its whitepaper.

Bridge Zero replaces neither: it is a building block either could use. On the BNB Smart Chain testnet since Nov 1.

Business model: who pays, what it costs

FOR POT holders · EVM apps and wallets · Portaldot · Liquidity providers

REVENUE, PLANNED (the testnet charges no fee)

Bridge fee	**0** to **5** bps a transfer, in v2 contracts
Instant fee	LPs quote **5** to **30** bps; the protocol keeps **10** to **20%** (est.)
Network fee	at cost, plus a margin
Light-client service	per integrator reading proven Portaldot state
Grants	ecosystem programs

COST TO RUN (est.: 1 gwei, ETH at $3,000)

	ETHEREUM	AN L2
A proven mint 2,391,069 gas	about $7.20 a mint	cents a mint
Following V3's validator set about 9.6M gas a day	about $870 a month	under $10 a month

Hourly head imports, optional since a mint brings its own header, would add about 45M gas a day (est.).

At **$1M** a month in volume, about **$620** a month in fees; at **$10M**, about **$6,200** (est.). Until then: grants, prizes and the light-client service. L2s first.

Scenario: a 5 bps bridge fee on all volume; 40% choose Instant at 20 bps, the protocol keeping 15%. A hosted proof API is not built yet.

Roadmap: narrow the trust, then mainnet

DONE
Oct 14 - Nov 1

- Light client and bridge, both ways, on four EVM testnets
- BNB Smart Chain testnet, the fourth, on Nov 1

→ the test network of the chain Portaldot's own bridge serves

MONTH 1
Nov 10 - Dec 9

- LP fast-fill on the way back
- The proof kit as an npm package and a CLI
- Threat model and audit preparation
- Grant applications

→ the vault pays only after finality; the LP carries the wait and the reorg risk

MONTH 2
Dec 10 - Jan 9

- v2 vault and minter: a bridge fee, fast-fill on the way in
- BEEFY verifier prototype
- A light client of Portaldot's V3 mainnet
- Base mainnet contracts, no value yet

→ cheaper imports; the same proofs on V3 mainnet

MONTH 3
Jan 10 - Feb 9

- Outside audit
- Admin behind a timelock or multisig
- Independent committee keys
- Guarded mainnet with caps
- 2–3 LPs · 1–2 pilot wallets or apps

→ a constrained admin, no single operator, audited code

AFTER FEB 9
not scheduled yet

- A proven way back: an Ethereum light client on Portaldot

→ no trusted committee

- Plans, not promises: a step counts once it runs on chain. V3's validators and sudo key stay trusted.

Bridge Zero

Team, and the ask

THE ASK

Grants toward the outside audit, independent committee members, 1–2 pilot wallets or apps, and 2–3 liquidity providers.

WHY KEEP BACKING IT

- It runs both ways between the Portaldot V3 testnet and four EVM testnets, the way back through a trusted committee: **149** transactions anyone can open.
- It is built on Portaldot: its finality, its EVM and its state, as a bridge and a proof kit.
- It is checkable: CI on every push, a Verify page, a guide to deploy your own copy.
- It has a dated plan that narrows the trust step by step.

TEAM

Jonathan Moore

Builder · github.com/jonathan-moore58

Abu Fauzan

Demo & presenter · abk970

APP	jonathan-moore58.github.io/bridge-zero/app
STATUS	jonathan-moore58.github.io/bridge-zero
CODE	github.com/jonathan-moore58/bridge-zero
X	x.com/BridgeZero1